



Amrita School of AI
COIMBATORE CAMPUS

UNIT 2 · SESSIONS 8-13

OpenMP

Using every core on one node, and the three bugs that will bite you.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

Fork and join



Directives the compiler reads.

Without `-fopenmp` they are **ignored** and you get a correct serial program.

Same source, either way.

A parallel region replicates. It does not divide.

```
#pragma omp parallel
{
    int id = omp_get_thread_num();
    printf("thread %d\n", id);
} // implicit barrier
```

With 8 threads this prints **8 times**. `parallel` alone duplicates work.

`for` is what divides it

```
#pragma omp parallel for  
for (int i = 0; i < n; i++)  
    a[i] = b[i] + c[i];
```

Loop-carried dependence

- **Wrong**

```
#pragma omp parallel for  
for (i = 1; i < n; i++)  
    a[i] = a[i-1] + b[i];
```

Iteration i needs $i-1$. OpenMP does not check. The answer is simply wrong.

- **Your job**

OpenMP parallelises whatever you point it at.

Checking for dependences is not the compiler's job. It is yours.

Every variable is shared or private

CLAUSE	MEANING
<code>shared</code>	one copy (default)
<code>private</code>	own copy, uninitialised
<code>firstprivate</code>	own copy, initialised
<code>lastprivate</code>	copied back out
<code>reduction</code>	private, then combined

USE THIS ALWAYS ``default(none)`` The compiler then refuses to guess, and a forgotten variable is a ****compile error**** instead of a silent race.

Shared accumulator

● Race

```
double sum = 0;  
#pragma omp parallel for  
for (...) sum += a[i];
```

Read, add, write. Two threads interleave, one update is lost. Wrong, quietly, and differently each run.

● Reduction

```
#pragma omp parallel for \  
    reduction(+ : sum)  
for (...) sum += a[i];
```

Private copy per thread, combined once at the end.

NOT THE SAME THING

`critical` gives the right answer and terrible performance

- **critical**

Every iteration takes a lock.

The loop is fully serialised **plus** lock overhead.

Reliably slower than serial code.

- **reduction**

No per-iteration synchronisation at all.

One combination step of $\log(\text{threads})$ at the very end.

The third bug is the interesting one.

The answer is correct. There is no race. And it runs slower than serial.

Different variables, one cache line

```
double partial[8];  
#pragma omp parallel  
partial[id] += x;
```

8 doubles = **64 bytes** = one cache line.

Every write by any thread invalidates the line in **every other core**.

The line ping-pongs across the interconnect.

GETS WORSE WITH MORE THREADS

Three fixes, best last

- Pad

```
struct { double v;  
        char pad[56]; }  
p[8];
```

One cache line each.

- Accumulate locally

```
double local = 0;  
for (...) local += a[i];  
partial[id] = local;
```

One write, not a billion.

- Just use reduction

```
reduction(+ : sum)
```

It already does exactly this.

`schedule` is one word that can double your efficiency

SCHEDULE	USE WHEN
<code>static</code>	equal work per iteration
<code>dynamic, k</code>	uneven or unpredictable
<code>guided</code>	moderately uneven

Triangular loop, `static`: thread 0 does ~7x the work of thread 3.

Everyone waits. $E \approx 0.4$.

Change to `dynamic, 16`: $E > 0.9$.

Tasks, for recursion and trees

```
#pragma omp task  
psort(a, lo, mid);  
#pragma omp task  
psort(a, mid, hi);  
#pragma omp taskwait
```

A task costs $\sim 1 \mu\text{s}$ to create, so **always** stop recursing at a cut-off.

PUTTING IT TOGETHER

The serial fixes came first, and were worth 13x

VERSION	TIME	SPEEDUP
naive ijk	42 s	1x
reordered ikj	9.8 s	4.3x
blocked, BS=64	3.1 s	13.5x
+ 16 threads	0.24 s	175x
cuBLAS-class BLAS	0.11 s	380x

Parallelising the ****naive**** version first would have given you 16 cores all waiting on memory. Fix the memory behaviour, then add cores. They multiply.

And then call a library anyway.

A tuned BLAS still beats your best hand-written kernel by 2x.

Next: when one node is not enough

MPI, and the deadlock everybody writes.